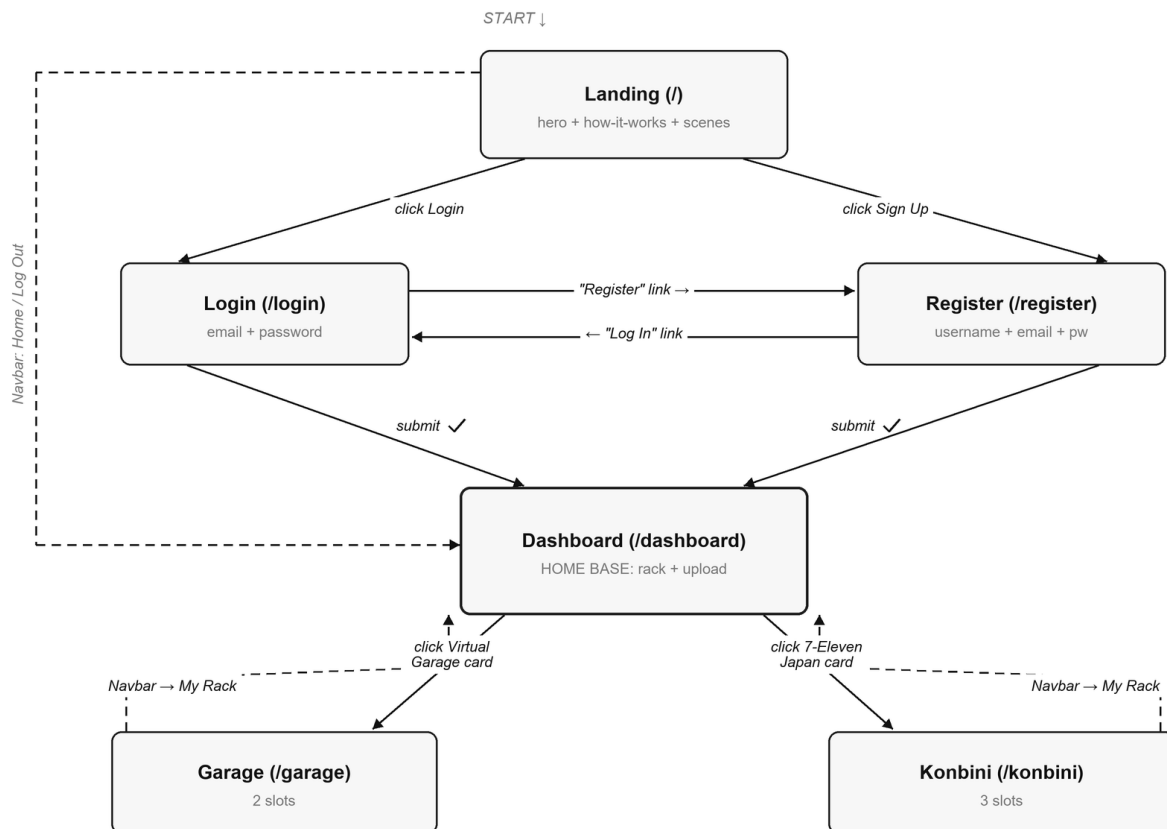


Wireframes & Component Breakdown

Boxes and labels only; no colour or typography decisions here, those come later. Diagrams below are generated from the actual PixelRack client (its routes, **Navbar**, **DashboardPage** and **ScenePage** components) rather than sketched from the template, so the boxes match what the app really renders today.

Step A: Screen map



Dashed = Navbar links (Home / Log Out / My Rack), available on Dashboard, Garage & Konbini alike.

Screen map: boxes = routes, arrows = what the user clicks

What is the first screen the user lands on? Landing (/): the hero with the PixelRack emblem, the “How It Works” steps and the scene previews.

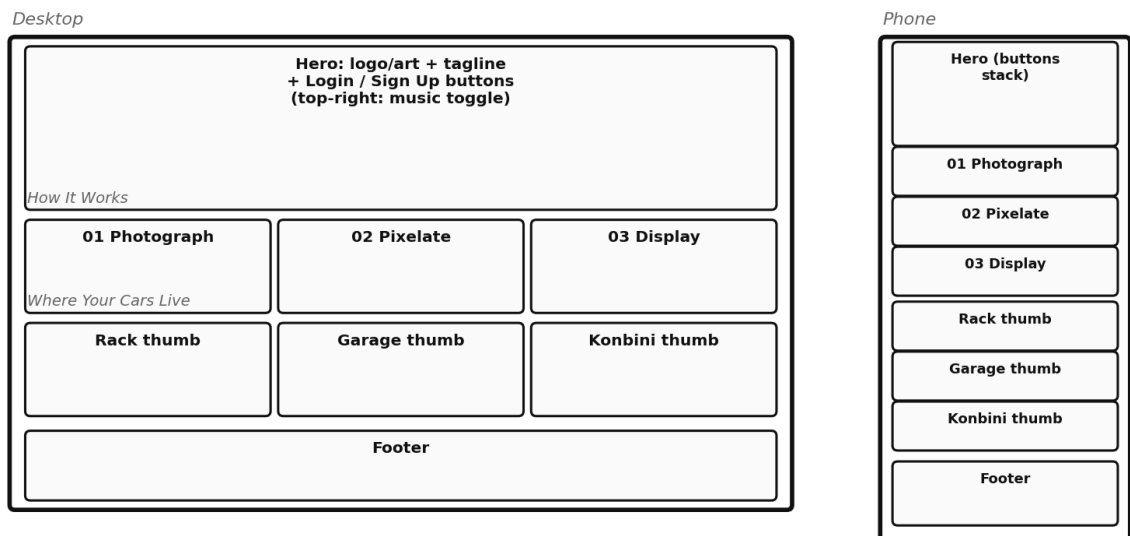
Is there a “home base” most others return to? Yes: Dashboard (/dashboard). It's where a successful login/register lands, and the shared **Navbar** (present on Dashboard, Garage and Konbini) always has a “My Rack” link back to it. That's the app's nav.

Any screen with no way back? Nothing is fully dead-ended, but one rough edge is real: Login and Register have no **Navbar** and no labelled “Back” link; the only way to Landing is clicking the PixelRack logo image at the top of the form. It works, but it isn't discoverable as a nav action the way a text link would be. Worth a small “← Back to Home” link if there's time; not worth blocking on.

Step B: One box-sketch per screen

Four sketches, not six: Login/Register share one mirrored layout, and Garage/Konbini share one **ScenePage** template with different artwork and slot counts, the same grouping the app proposal already used, because they really are the same component.

Screen 1: Landing (/)



Screen 1: Landing (/)

Screen 2: Auth, Login (/login) & Register (/register), mirrored layout

Desktop

Logo (→ /)

Card: heading
error slot
[Username (Register only)]
Email
Password
Submit button
"switch form" link

Note: this screen is already a centred, narrow card at every width; no breakpoint needed.

Phone

Logo (→ /)

Card: heading
error slot
[Username (Register only)]
Email
Password
Submit button
"switch form" link

Screen 2: Auth, Login (/login) & Register (/register)

Screen 3: Dashboard / Rack (/dashboard)

Desktop

Navbar: logo | Home | My Rack | Upload ... Profile | Log Out

Upload form (sidebar)
file drop + crop
name / series
submit + progress

Rack header: name, counts, sort, filter

Rack grid: shelves of car tiles
(grouped by series)

Environment gallery: 3 cards
(Rack / Garage / Konbini)

Footer

Phone

Navbar (wraps)

Upload form
(now full width,
stacks above rack)

Rack header (stacks)

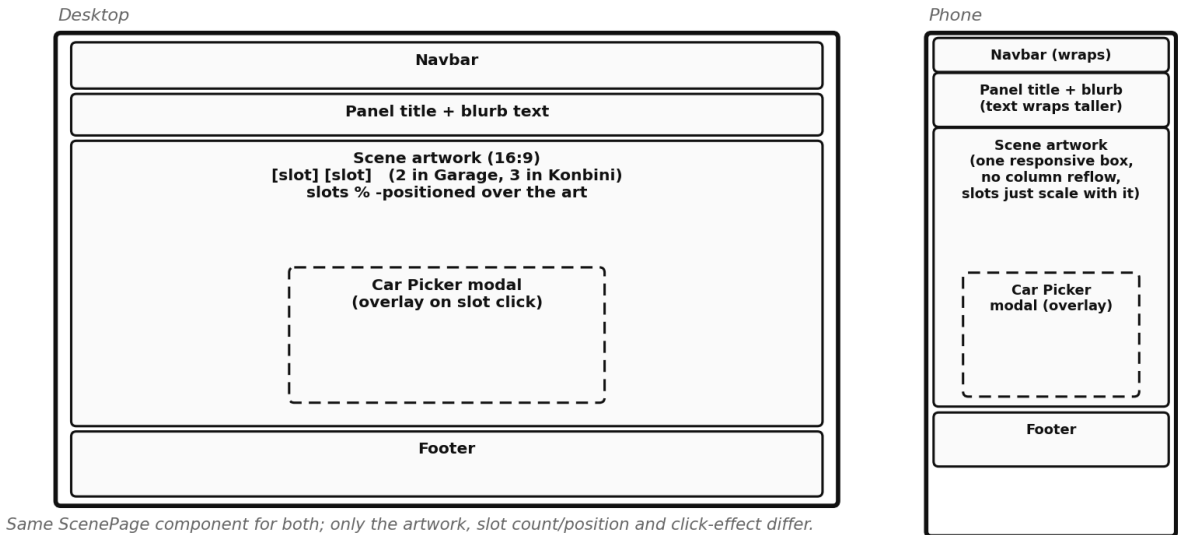
Rack grid: tiles
shrink, rows
stay 9-wide

Env. gallery:
3 cols → 1 col

Footer

Screen 3: Dashboard / Rack (/dashboard)

Screen 4: Scene template: Garage (/garage) & Konbini (/konbini)



Screen 4: Scene template, Garage (/garage) & Konbini (/konbini)

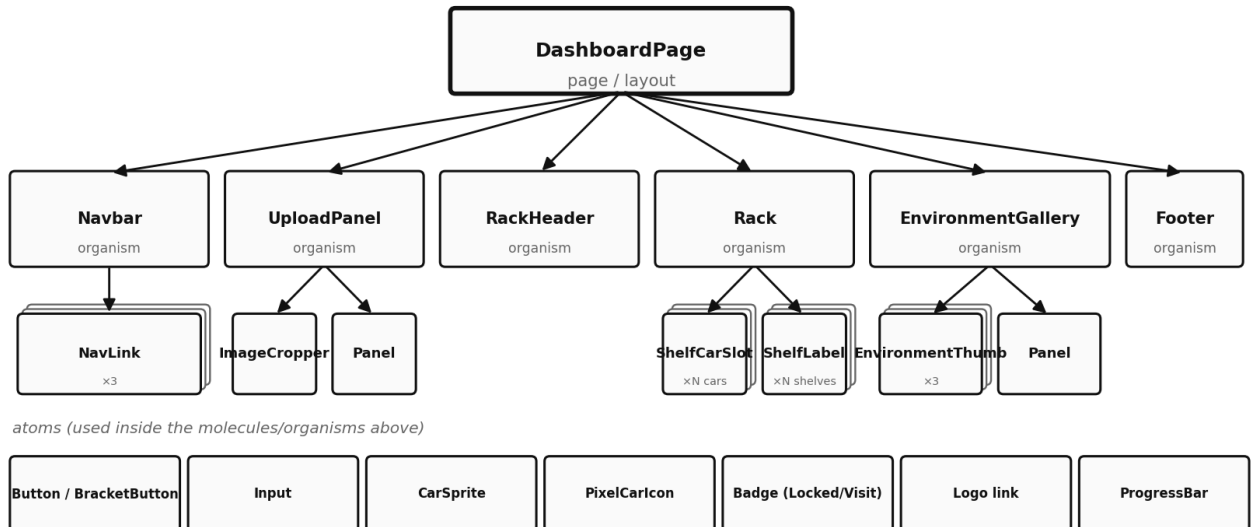
Same layout facts as a table, for quick reference:

Screen	Desktop layout	Phone layout (what stacks)	Navigates to
Landing (/)	Hero + 2 rows of 3 cards + footer	Everything stacks to a single column	Login, Register (or straight to Dashboard if signed in)
Auth (/login, /register)	Centred narrow card	Same card, no reflow needed	Dashboard (on submit); the other auth screen (via link)
Dashboard (/dashboard)	Navbar + 320px upload sidebar + main column	Navbar wraps; sidebar becomes full width and stacks above the rack; gallery 3 cols → 1	Garage, Konbini (via gallery cards); Landing (via Navbar)
Garage (/garage)	Navbar + title/blurb + 16:9 scene (2 slots)	Same single responsive scene box, no column reflow	Dashboard (via Navbar)
Konbini (/konbini)	Navbar + title/blurb + 16:9 scene (3 slots)	Same single responsive scene box, no column reflow	Dashboard (via Navbar)

Step C: Component tree

Busiest screen: **Dashboard**, since it's the only one that both reads and writes data (upload, sort, filter, delete) rather than just placing cars in slots.

Component tree: Dashboard / Rack (busiest screen)



Arrows above show the parent → child relationship. xN boxes mark a component that repeats and is rendered from a list (built once, given a unique key per item).

Level	What it is	Your components
Atoms	smallest pieces	Button, BracketButton, Input (bare <input>), CarSprite, PixelCarIcon, status badge (Locked/Visit/slots), logo Link, ProgressBar fill
Molecules	small groups of atoms	Panel (header bar + body slot, reused everywhere), NavLink (icon + label), ShelfCarSlot (sprite + delete button + name tooltip), EnvironmentThumb (art + name + badge), ImageCropper
Organisms	whole sections	Navbar, UploadPanel, RackHeader, Rack, EnvironmentGallery, Footer, and ScenePage (the Garage/Konbini organism, with CarPickerModal as an overlay it opens)
Page / layout	the screen that arranges organisms	DashboardPage (Dashboard); GaragePage / KonbiniPage (each just configures ScenePage with its own artwork + slots)

Sanity-check the two rules:

- **A component that repeats is a real component.** `ShelfCarSlot` renders once per car (keyed by `car.id`), `EnvironmentThumb` once per environment, `NavLink` once per nav entry; all marked $\times N$ in the tree above, all built once and rendered from a list.
- **A level uses the levels below it, never above.** `Rack` (organism) renders `ShelfCarSlot` (molecule) which renders `CarSprite` (atom); never the reverse. The one component that appears in two organisms is `Panel` (used by both `UploadPanel` and `EnvironmentGallery`), which is fine: a molecule can be reused by more than one organism, it just can't reach back up.

Step D: Sanity check

Most important task: **upload a car, then place it in the Garage.** Walking it screen by screen:

1. Landing (/) → click **Login** → Login screen.
2. Submit email + password → redirected to **Dashboard**; `cars` and `environments` load.
3. Fill the **Upload form** in the sidebar (photo, crop, name, series) → submit → new car appears in the **Rack grid** (optimistic add via `onUpload`).
4. Click the **Virtual Garage** card in the Environment gallery → Garage screen.
5. Click an empty slot → **Car Picker modal** opens (an overlay, not its own route).
6. Pick the new car → modal closes → it appears in the Garage slot.
7. Click the car → plays the wash effect (shine + bubbles).
8. Click **My Rack** in the Navbar → back to Dashboard.

Did I hit a screen I forgot to sketch? Yes, once: the **Car Picker modal**. It isn't a route, so it doesn't get its own Step B sketch, but it's a real, self-contained piece of UI; it's called out as an overlay on the Scene wireframe (Screen 4) and sits in the component tree conceptually alongside `ScenePage`, even though the tree above focuses on Dashboard.

Did any navigation have nowhere to go? No hard dead end. The one soft spot is the Login/Register → Landing path noted in Step A (logo click only, no labelled link).

Does every piece of state from the proposal have an owning component? Yes, checked against the proposal's state table: `cars`, `environments`, `sort`, `seriesFilter`, `isLoading`, `error` all live in `DashboardPage`; the upload form's own fields (`file`, `previewUrl`, `crop`, `name`, `series`, `progress`, `isProcessing`, `pixelationEnabled`) live in `UploadPanel` and only call back up once, via `onUpload`; `placements`, `pickingSlot`, `activeEffect` live in `ScenePage`. Nothing is missing an owner.

What to keep

- The screen map above → becomes the `<Routes>` in `App.jsx` and the links in `Navbar` (both already built, so this documents what exists).
- Each labelled box in Step B → a component under `client/src/components/` or `client/src/pages/`, already sorted into the atomic levels in Step C.
- The “what stacks on a phone” notes → the Tailwind breakpoints already in use (`sm:`, `lg:`, `flex-wrap`), confirmed against the real classNames, not guessed.